

The two-board assembly guide

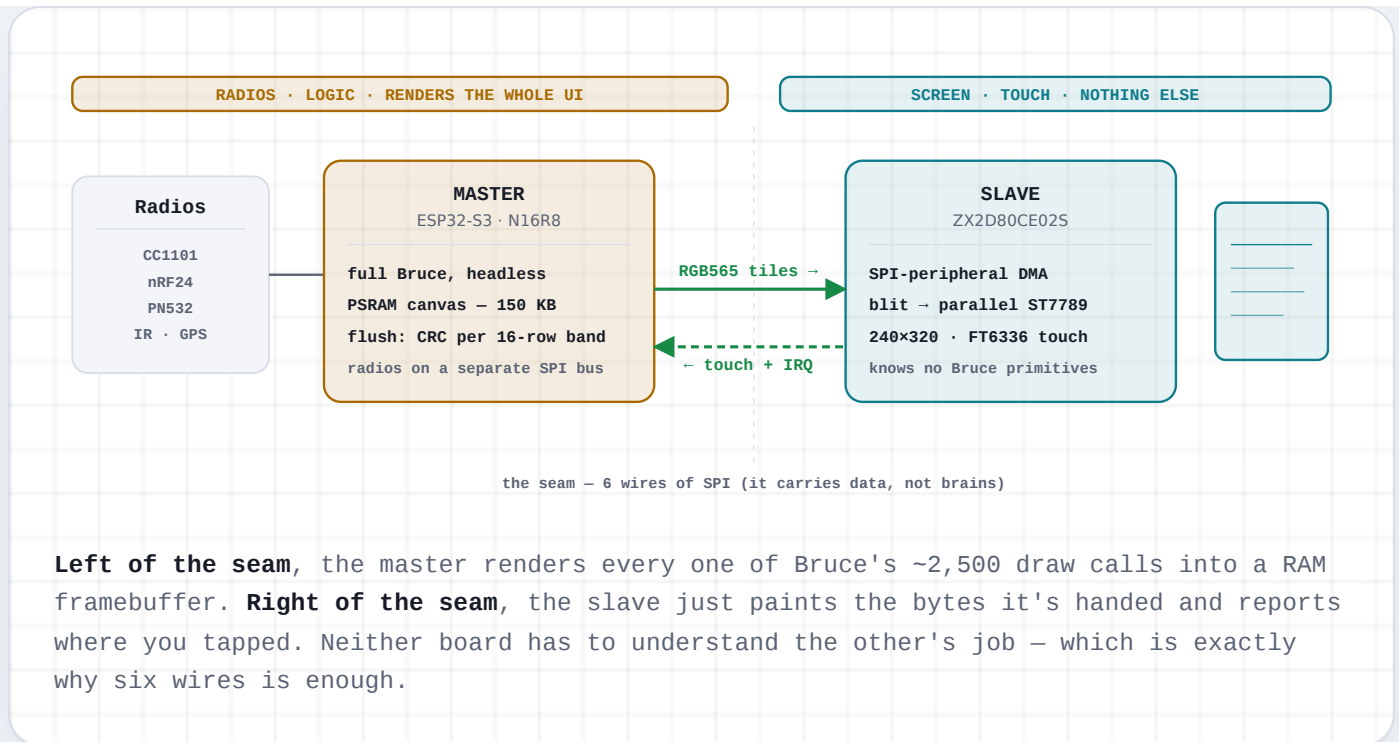
One box couldn't hold both a parallel screen and a stack of radios. So Bruce split up.

Irreconcilable GPIO differences. The ZX2D80CE02S drives its display over an 8-bit parallel bus that claims ~13 pins — nothing left for the radios. The fix: give every radio to a **master** ESP32-S3 with pins to spare, and demote the display board to a dumb pixel blitter. The master keeps the radios and runs **full Bruce, headless**; the **slave** keeps the screen. They still talk — over six wires of SPI.

● master: **radios + full Bruce** ● slave: **ZX2D80CE02S** display only ● **6-wire SPI @ 20 MHz**
● pixel-perfect **PSRAM canvas** ● portrait **240×320** (v1)

01 The split

Two ESP32-S3 boards, one Bruce. The master owns every radio and every line of Bruce's logic; the slave owns the panel and the touch controller. The only thing between them is a thin SPI link that carries pixels one way and touch the other.



02 What you'll need

Two ESP32-S3 boards and the radios you wanted all along. Everything but the display board hangs off the master.

PART	ROLE	NOTES
Master board	Runs full Bruce, headless	Any ESP32-S3; N16R8 recommended (16 MB flash / 8 MB PSRAM). PSRAM is mandatory — the canvas lives there.
Slave board	The display + touch	Panellan ZX2D80CE02S (aka SC05_X / "WT32S3-28S PRO"). WT32-S3-WROVER N8R2, parallel ST7789 240×320, FT6336 cap-touch. Already Bruce-ported.
CC1101	Sub-GHz RF	On the master's radio SPI bus.
nRF24L01+	2.4 GHz	Shares the radio SPI bus; its own CS + CE.
PN532	NFC / RFID	On the master's I2C bus (@0x24). Costs no extra GPIO.
7× jumpers	The link harness	SCLK, MOSI, MISO, CS, IRQ, GND, 3V3. Keep them short.
GPS · IR · μSD	Optional extras	All on the master's remaining GPIO.

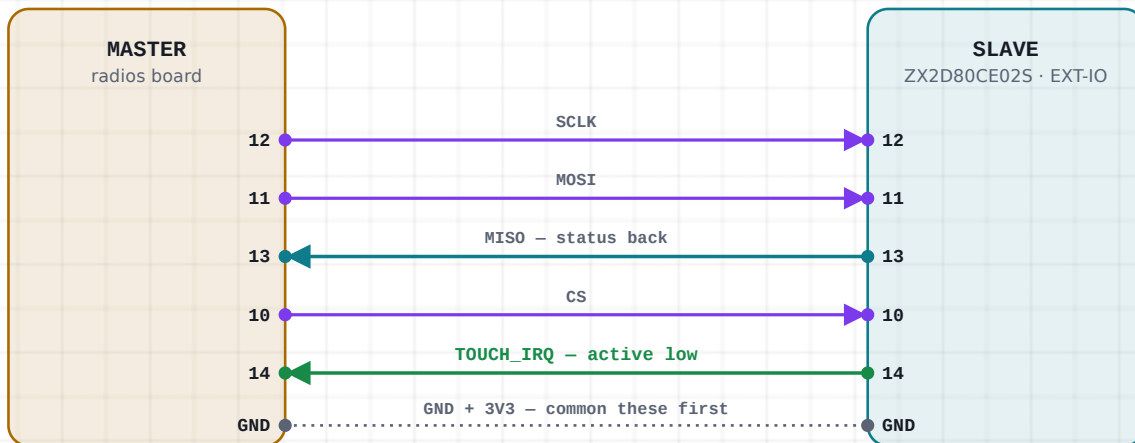
NO PSRAM, NO PICTURE

The canvas is a full-screen `TFT_eSprite` — $240 \times 320 \times 2 = \mathbf{150\ KB}$ — allocated with `createSprite()` in PSRAM. An N8R2 master (2 MB) is enough; N16R8 just leaves the most headroom for Bruce itself (the firmware is ~ 3.75 MB and uses $\sim 48\%$ of internal RAM before the canvas).

03 Wire the link – the custody arrangement

Six signals plus power. The defaults use the **same GPIO number on both ends**, so the harness is symmetric: 12→12, 11→11, and so on. On the slave these are the EXT-IO header pins (10–14) — the only general IO the parallel panel leaves free.

SIGNAL	MASTER GPIO	SLAVE GPIO	DIRECTION
SCLK	12	12	master → slave
MOSI	11	11	master → slave
MISO	13	13	slave → master
CS	10	10	master → slave
TOUCH_IRQ	14	14	slave → master · active-low
GND	GND	GND	common – wire first
3V3	3V3	3V3	shared supply



Same number, both ends. Purple wires are master-driven (SCLK, MOSI, CS); cyan is the slave clocking its status block back on MISO; green is the touch interrupt the slave pulls low when a tap is waiting. The master runs this on **HSPI, mode 0, MSB-first, 20 MHz**; the slave listens on `SPI2_HOST`.

GROUND BEFORE SIGNAL

Common the grounds (and share the 3V3 supply) **before** you land the signal wires — a floating ground between two SPI devices is the fastest route to a dead first-light. The master holds IRQ up with an internal pull-up; the slave drives it low only when `seq` has advanced. Both ends are configurable: master in `boards/bruce-remote-master/pins_arduino.h`, slave in `slave_zx2d80ce02s/src/main.cpp`.

04 Wire the radios to the master

The whole reason for the divorce. Every module below wires to the master's own GPIO — the slave has no idea any of it exists. The radio SPI bus (GPIO 4/5/6) is separate from the display link bus, so pixels and packets never contend.

MODULE	BUS	MASTER PINS
CC1101	radio SPI	SCK 4 · MOSI 5 · MISO 6 · CS 7 · GD00 15
nRF24L01+	radio SPI (shared)	CS 16 · CE 17 · SCK/MOSI/MISO shared 4/5/6
PN532	I2C	SDA 8 · SCL 9 (@0x24)
GPS	UART	TX 18 · RX 21
IR TX / RX	GPIO	TX 1 · RX 2 (also 15/16/17)
microSD	radio SPI	CS -1 by default – set to your wiring
BOOT button	GPIO	0 – doubles as Bruce's Select

WHY THESE PINS

The map dodges the octal flash/PSRAM range (GPIO 26–37) an N16R8 uses internally, the native-USB pins (19/20), and UART0 (43/44). Two dummy TFT pins are also defined so TFT_eSPI still compiles — they are **never driven**; `tft.init()` allocates the canvas instead of touching a panel.

05 Flash both boards

Two firmwares, two PlatformIO projects. Flash the **slave first** so it's already listening on SPI when the master boots and immediately starts streaming its first full frame.

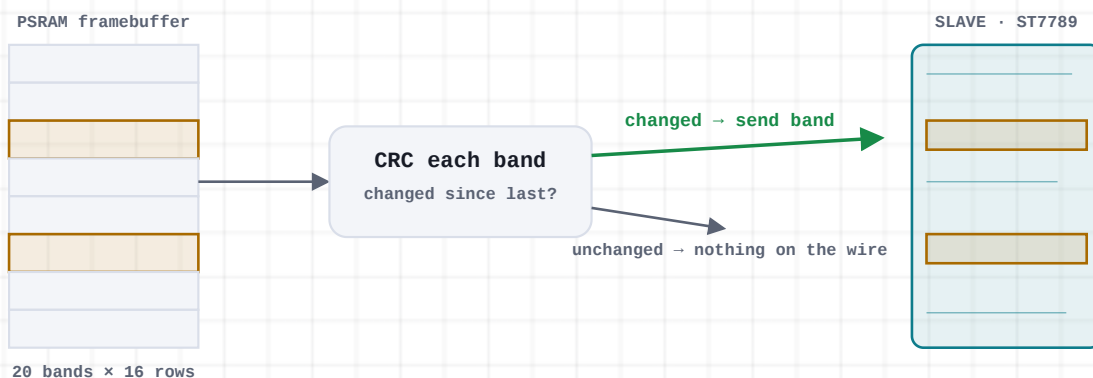
```
# 1 · SLAVE – the display board (standalone LovyanGFX blitter, none of Bruce's logic)
cd remote_display/slave_zx2d80ce02s
pio run -t upload # → firmware ~441 KB · Flash 13% · RAM 10%
```

```
# 2 · MASTER – the radios board (full Bruce, headless PSRAM canvas)
pio run -e bruce-remote-master -t upload # → firmware ~3.75 MB · Flash 22%/16MB
```

The slave project is standalone — it isn't part of the Bruce build and shares exactly one file with the master, `bruce_remote_link.h`. The master is a normal Bruce board target. What's left is on-hardware bring-up (below); the SPI timing and color order simply can't be proven at compile time.

06 How the picture gets down six wires

A full 240×320 UI over a single SPI link sounds slow. It isn't — because a still screen sends nothing at all. The master ships only the pixels that actually changed.



Idle screen → 0 bytes. A background task on core 0 CRC-checks all 20 bands every ~20 ms and ships only the ones that moved (a 16-byte header, then that band's RGB565). A full redraw is ~150 KB (~75 ms at 20 MHz); a typical partial update is sub-millisecond. Because **real pixels** are rendered on the master, the slave never parses a font, sprite, JPEG or QR — it just blits bytes, which is what makes it pixel-perfect.

07 First light — bring-up in order

Each milestone unlocks the next. Everything here needs both boards on the bench — none of it can be proven at compile time, so budget a session with a logic analyzer handy for step 1.

1 SPI pixel path

Send a header + one band and confirm the slave blits it. Tune the header→payload gap (`REMOTE_LINK_GAP_US`, 40 μs default) so the slave's DMA re-arms in time, and check color order with a known test tile — flip `rgb_order` / `setSwapBytes` if red and blue swap.

2 Boot Bruce on the canvas

Let the master run; the first flush is full-frame. The Bruce menu should appear on the slave panel.

3 Touch

Tap the screen: the slave raises IRQ, the master polls over the link, and the menu should respond. Verify the one-event `seq` handshake and that coordinates land where you tapped.

4 Dirty-band tuning

Watch flush cadence vs. CPU; once it's stable, raise `REMOTE_LINK_HZ` past the initial 20 MHz.

5 Radios

Wire a module and run an app end-to-end — the payoff, and the whole point of splitting.

6 Backlight & sleep

Confirm the `BACKLIGHT` and `SLEEP` control frames dim, wake, and blank the panel from the master.

08 The fine print

Two known v1 limitations, neither fatal — both honest consequences of rendering on one board and blitting on the other.

PORTRAIT ONLY

v1 is pinned to **240×320** so the canvas dimensions equal the slave's native panel. Landscape rotation is deferred; the `ROTATION` control frame is wired but the canvas keeps portrait geometry.

PUSHSPRITE() COMPOSITES NOWHERE

Sub-sprites pushed onto TFT_eSPI's (headless) parent panel don't reach the canvas. The only core user is the `tururururu.cpp` shark easter-egg; the **entire real UI draws straight to `tft`** and is unaffected.

STATUS

The wire protocol is host-compiled and self-tested (struct sizes, checksums, corruption). Both firmwares compile clean. Runtime behavior — SPI-slave DMA re-arm timing, color order, and the touch handshake — is **inherently on-hardware** and is what the milestones above verify.

Divorced Bruce · split-display build — the master keeps the radios, the slave keeps the screen.

6-wire SPI seam · pixel-perfect PSRAM canvas · dirty-band streaming · portrait 240×320 (v1)